**The Characters of Zelda site is a data-driven, interactive, and responsive website that pulls character entries from the database and organizes them into clean, easy-to-browse sections. It uses Python to sort characters into categories, group incarnations under the same name, and decide what should appear on the page. Django's templating system then turns that data into features like slideshows, tabs, and scrollable galleries.**

**Python Concepts Used in Home Template:**

The two slide shows (Featured Characters and More Featured Characters) and the two slides (More Characters and Fan Characters) use the following python concepts:

**Data Grouping** - grouping multiple incarnations of the same character under one key.

**Lists** - storing multiple character objects inside each group.

**Dictionaries** - mapping a character's name to its list of incarnations.

**Iteration** - looping through groups and characters to build slides dynamically.

**Conditionals** - deciding whether a character becomes a slideshow (multiple incarnations) or a single card (one incarnation).

**String Manipulation** - cleaning character names (e.g., removing spaces) to generate valid HTML IDs.

**Featured Characters Slides**

*In Zelda lore, many characters **appear in multiple incarnations** across different games. Each incarnation is a new version of the same character, with different designs, roles, and story contexts.*

*To represent this in code, I grouped these incarnations together and each slide is an item in that group. Each character name becomes a dictionary key, and all their incarnations are stored as a list under that key.*

*This grouping allows the homepage to loop through each incarnation using **iteration** and display them as a **slideshow**. Each tab is a group of one character, where each slide is their different form.*

When the user creates a featured character in the admin panel, when the user selects "section 1" Django saves that value in the database in the "Character" model. The view loads these characters from the DB based on the field section=1 and groups them by name, and into a dictionary with the name being the key and the properties being the values stored as a list. That dictionary is stored in memory.

Python Dictionary the view sends to the home page template:

```python
# Put in a dictionary for the template
content = {
    'section1_groups': section1_groups,
    'section2_groups': section2_groups,
    'more_characters': more_characters,
    'fan_characters': fan_characters,
}
```

section1_groups store the keys (character names) in groups.

To access these for the slides, I used a nested for loop, the outer loop has:

```
{% for name, group in section1_groups.items %}
```

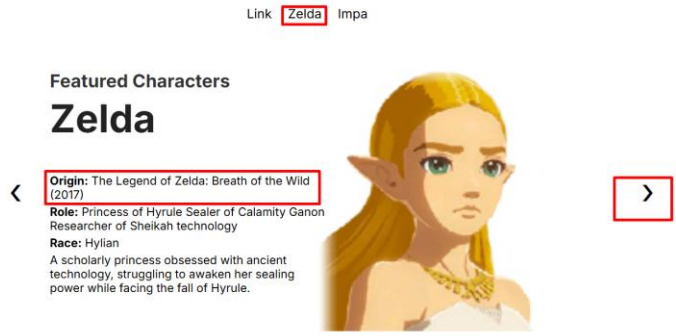
The whole loop runs once per character name, creates one slideshow per character, generates the slideshow container and arrows.

```
{% for character in group %}
```

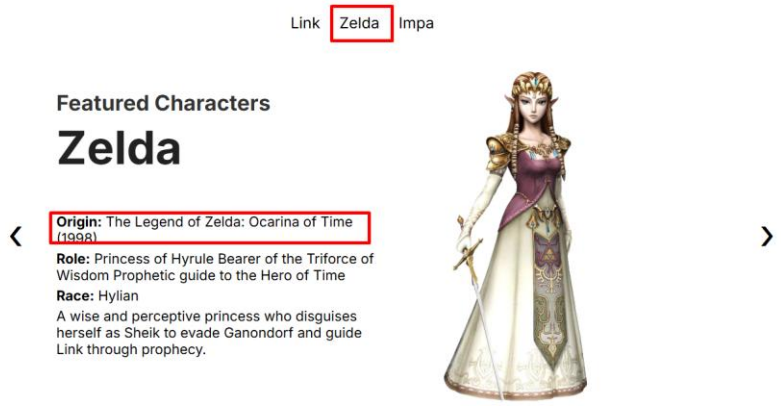
The inner loop runs once per incarnation, creates each individual slide for the group of characters, and displays the incarnation's name, race, origin, role, description, and image.

```
{% for character in group %}
<div class="slide">

    <div class="featured-row">

        <div class="featured-text">

            <div class="featured-heading-block">
                <h2 class="featured-section-title">Featured Characters</h2>
                <h3 class="featured-name-top">{{ character.name }}</h3>
            </div>

            <p><strong>Origin:</strong> {{ character.origin }}</p>
            <p><strong>Role:</strong> {{ character.role }}</p>
            <p><strong>Race:</strong> {{ character.race }}</p>
            <div>{{ character.description }}</div>

        </div>

        <div>
            <img src="{{ character.image_url }}" class="featured-image">
        </div>
    </div>
</div>
{% endfor %}
```

**Zelda incarnation 1:**




**Zelda incarnation 2:**




This same concept is used for both featured section 1 and featured section 2 (labeled more featured characters).


For the more characters section the user starts by first adding a character in the admin panel and selecting “more characters list”. It’s then loaded from the database (Python looks for the

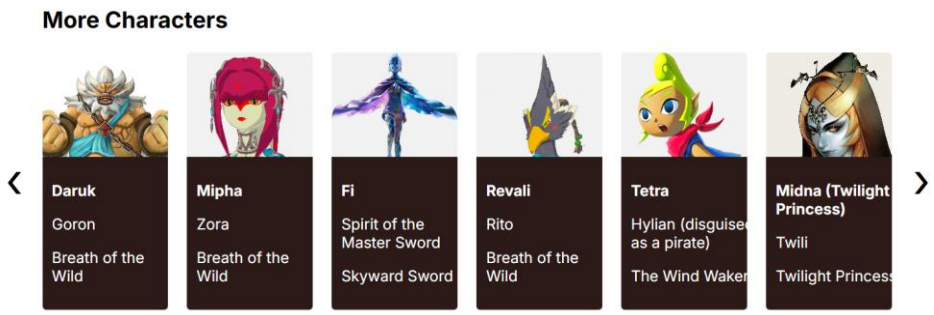
characters that have “more characters list” selected and who do not have featured section selected.)  This results in a query set which is a python list-like object. The query set is passed to the template through the following key-value pair in the content dictionary in views:

```
# Put in a dictionary for the template
content = {
    'section1_groups': section1_groups,
    'section2_groups': section2_groups,
    'more_characters': more_characters,
    'fan_characters': fan_characters,
}
```

The query set gets sent to the template to loop through as “{% for character in more_characters %}”
.

```
<div class="list-row" id="more-row">
    {% for character in more_characters %}
        <div class="character-card"
             onclick="showDetails(
                 '{{ character.name }}',
                 '{{ character.race }}',
                 '{{ character.origin }}',
                 '{{ character.role }}',
                 '{{ character.pk }}',
                 '{{ character.is_protected }}'
             )">

            {% if character.image_url %}
                <img src="{{ character.image_url }}" class="card-image">
            {% endif %}

            <div class="card-text">
                <p><strong>{{ character.name }}</strong></p>
                <p>{{ character.race }}</p>
                <p>{{ character.origin }}</p>
            </div>

        </div>
    {% endfor %}
</div>
```

As the loop iterates through each character it creates the horizontal scroll row:




The concept is the same for the fans section.

**Conditional Rendering:**

Another python concept used is conditions. In this same for loop, the following checks if a character image exists, if it doesn't hide images and show a different UI.

```
{% if character.image_url %}
    <img src="{{ character.image_url }}" class="card-image">
{% endif %}
```

**Template Inheritance:**

Each page uses Djangos template inheritance which allows the home, and all future pages that will be added, to reuse the navigation, footer, global css/js, and layout structure.

**Python Concepts used in views.py.**

**Data Filtering:**

**Python filtering is the process of selecting only the items that match certain conditions. In Zelda Characters app, Django's .filter() method is used to pull specific groups of characters from the database, such as Featured Characters, More Characters, and Fan Characters, based on the values stored in each character's fields.**

Zelda Characters site's views use data filtering to pull specific characters from the database based on conditions.

```python
# This function will show the Home page when requested
def home(request):  1 usage  Erica Millan
    # Get characters for Featured Section 1 where field is set to 1
    section1_chars = Character.Characters.filter(featured_section=1)
    # Get characters for Featured Section 1 where field is set to 2
    section2_chars = Character.Characters.filter(featured_section=2)

    section1_groups = group_by_name(section1_chars)
    section2_groups = group_by_name(section2_chars)

    more_characters = Character.Characters.filter(
        featured_section__isnull=True,
        more_characters_list=True
    )

    fan_characters = Character.Characters.filter(more_characters_list=False)
```

This helps the site to decide which characters belong in featured section 1 & 2 and the different character scroll rows.

Section 1 & 2 filtering – the code filters the database where featured section = 1 or 2 to determine which characters belong in the slide shows.

More & Fan Characters- the code filters the database where conditions for each are met to determine which characters belong in scroll rows, and which scroll row they belong.

**Data Grouping:**

**Python data grouping is the process of collecting related items under a shared key. In the Zelda Character site, the group_by_name() function groups multiple incarnations of the same character into a single dictionary entry, allowing the template to build one slideshow per character instead of treating each incarnation separately.**

The group by name function groups the characters by their name so the site can take multiple incarnations of the same character and put them in to one slide show.

```
def group_by_name(characters):  2 usages  Erica Millan
    groups = {}
    for char in characters:
        if char.name:
            key = char.name.lower().replace(" ", "-")
        else:
            key = "char-" + str(char.pk)
        if key not in groups:
            groups[key] = []
        groups[key].append(char)
    return groups
```

The section with the red box uses a dictionary to store the groups, list to hold the items in each group and iterates through each character to sort them.

So for:

Zelda (Ocarina of Time)

Zelda (Twilight Princess)

Zelda (Breath of the Wild)

The grouping will turn this into:

"zelda": [Zelda_OoT, Zelda_TP, Zelda_BotW]

So the template can build one tab for Zelda and one slide show with all of her incarnations.

**Dictionaries:**

**A Python dictionary stores data in key–value pairs. In this project, dictionaries are used to group character incarnations under a shared name and to package all homepage data into the content dictionary that the template receives. This makes it easy for the template to access and display the correct character groups and sections.**

group_by_name dictionary→ the dictionary uses the key as the characters name and value is a list of all incarnations of the character for the slide show and tabs.

Content dictionary → packages all of the homepage data.

**Lists:**

**A Python list is an ordered collection of items. In this project, lists are used to store groups of character incarnations and to loop through characters in the template. They make it easy to organize multiple objects together and build dynamic UI elements like slideshows and scrollable sections.**

**In group_by_name each character group is stored as a list:**

```
def group_by_name(characters):  2 usages  Erica Millan
    groups = {}
    for char in characters:
        if char.name:
            key = char.name.lower().replace(" ", "-")
        else:
            key = "char-" + str(char.pk)
        if key not in groups:
            groups[key] = []
        groups[key].append(char)
    return groups
```

So each key contains a list of all incarnations.

**Passing Structured Data Between Layers :**

**Python data flow describes how character data moves through the site: it starts in the database, is retrieved and organized by Python in the views, packaged into a dictionary, passed to the template, and finally rendered into the UI.**

All data starts in the database, in the Character Model. The views use filtering to retrieve the data based on specific conditions, and the result is a **QuerySet**, which behaves like a list of character objects. Python processed the data with grouping characters by name, sorting into sections, filtering based on fields, and building dictionaries to structure the data.

Transforming raw database rows into structured data the template can use.

```
def home(request):  1 usage  Erica Millan
    # Get characters for Featured Section 1 where field is set to 1
    section1_chars = Character.Characters.filter(featured_section=1)
    # Get characters for Featured Section 1 where field is set to 2
    section2_chars = Character.Characters.filter(featured_section=2)

    section1_groups = group_by_name(section1_chars)
    section2_groups = group_by_name(section2_chars)

    more_characters = Character.Characters.filter(
        featured_section__isnull=True,
        more_characters_list=True
    )
```

Python then packages the data into a dictionary

```
    # Put in a dictionary for the template
    content = {
        'section1_groups': section1_groups,
        'section2_groups': section2_groups,
        'more_characters': more_characters,
        'fan_characters': fan_characters,
    }
```

Django send the data into a template

```
    # return rendered home page
    return render(request, 'CharactersOfZelda/CharactersOfZelda_home.html', content)
```

HTML template can access the keys in the dictionary. In the html template the data is rendered with the forloop.

```
<!-- Slideshows -->
{% for name, group in section1_groups.items %}
<div class="featured-slideshow s1" id="s1-{{ name|cut:"_" }}" style="display:block;">

    <div class="slideshow-container" id="slideshow-s1-{{ name|cut:"_" }}">

        <!-- FIXED ARROWS -->
        <button class="arrow-btn prev"
                onclick="plusSlide('slideshow-s1-{{ name|cut:"_" }}', -1)">
            &#10094;
        </button>

        <button class="arrow-btn next"
                onclick="plusSlide('slideshow-s1-{{ name|cut:"_" }}', 1)">
            &#10095;
        </button>
```

**Python concepts used in Models**

**The Character model uses core Python concepts such as classes, attributes, methods, inheritance, and default values. It also uses Django's field types, choices, and model managers to define how character data is stored and accessed. Together, these Python features create the structure that for all character data throughout the site.**

**Classes:**

A class is a blueprint for creating objects. The whole model is a Python class. Each character in the database becomes an instance of this class.

**Attributes:**

Each field becomes an attribute for the character object.

**Data types:**

The model uses different data types (charfield, textfield, Boolean field etc.)

Methods:

There are methods defined on the class. Pythons dunder method controls how the object is displayed. (in the admin panel)

```
def __str__(self):  Erica Millan
    return self.name
```

Inheritance: The Character class inherits from Djangos base model, giving the class database integration, field handling, validation, and ORM features.

```
class Character(models.Model):  11 usages  Erica Millan
    name = models.CharField(max_length=100)
    race = models.CharField(max_length=100)
    origin = models.CharField(max_length=100)
    description = models.TextField()
    role = models.CharField(max_length=100)
    abilities = models.TextField()
    image_url = models.CharField(max_length=500, blank=True, null=True)
    is_protected = models.BooleanField(default=False)
    more_characters_list = models.BooleanField(
        default=True,
    )
    has_slides = models.BooleanField(default=False)
```

**Python concepts used in CharacterForm**

**The CharacterForm uses Python concepts such as classes, inheritance, nested classes, lists, dictionaries, and keyword arguments to build a form based on the Character model. These Python structures control which fields appear in the UI, how they are displayed, and how user input is validated and saved.**

**Classes:**

The character form is defined by a python class, which acts as a blueprint to create form objects. Each time the form is displayed or submitted, Python creates an **instance** of the character form class.

**Inheritance:**

Character form inherits from ModelForm, giving it automatic field generation, built-in validation, integration with the Character model.

**Nested Classes:**

Meta class is nested in the form. This nested class is used to store configurations.

Attributes:

In the nested meta class attributes are defined to control how the form behaves and what it displays.

**Lists**

Fields attribute is a python list.

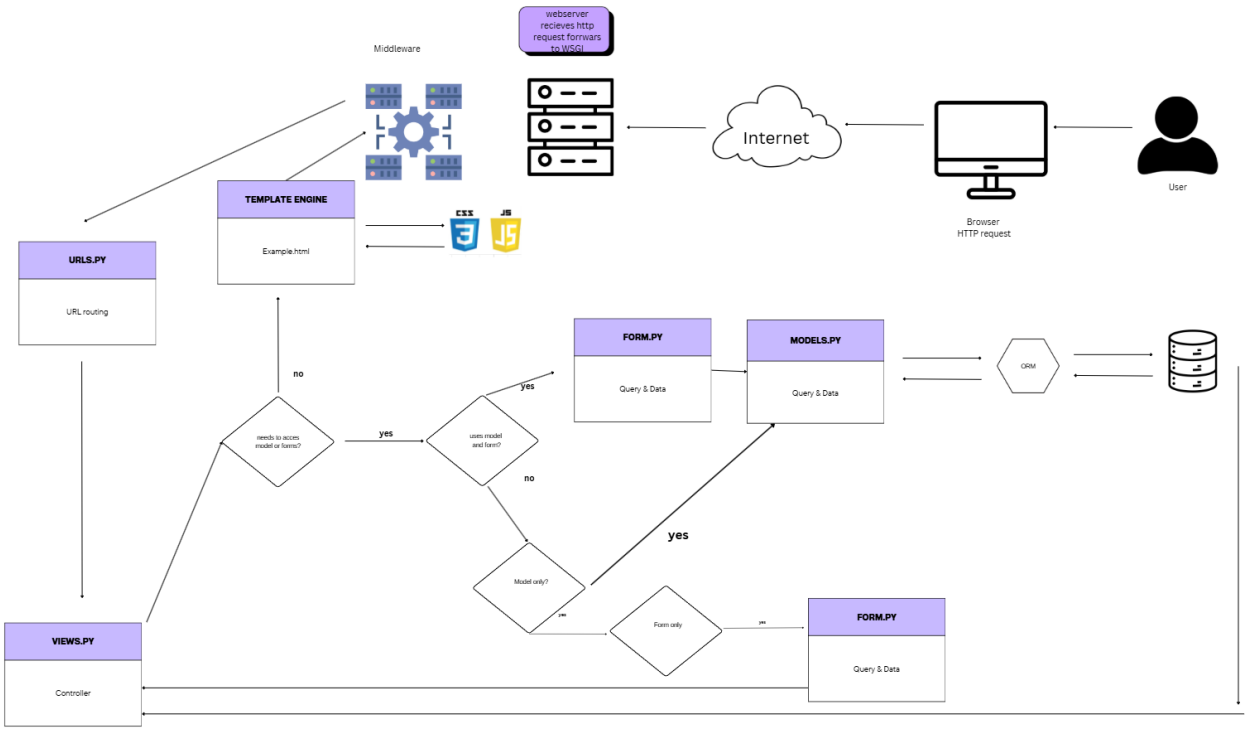
**Dictionaries:**

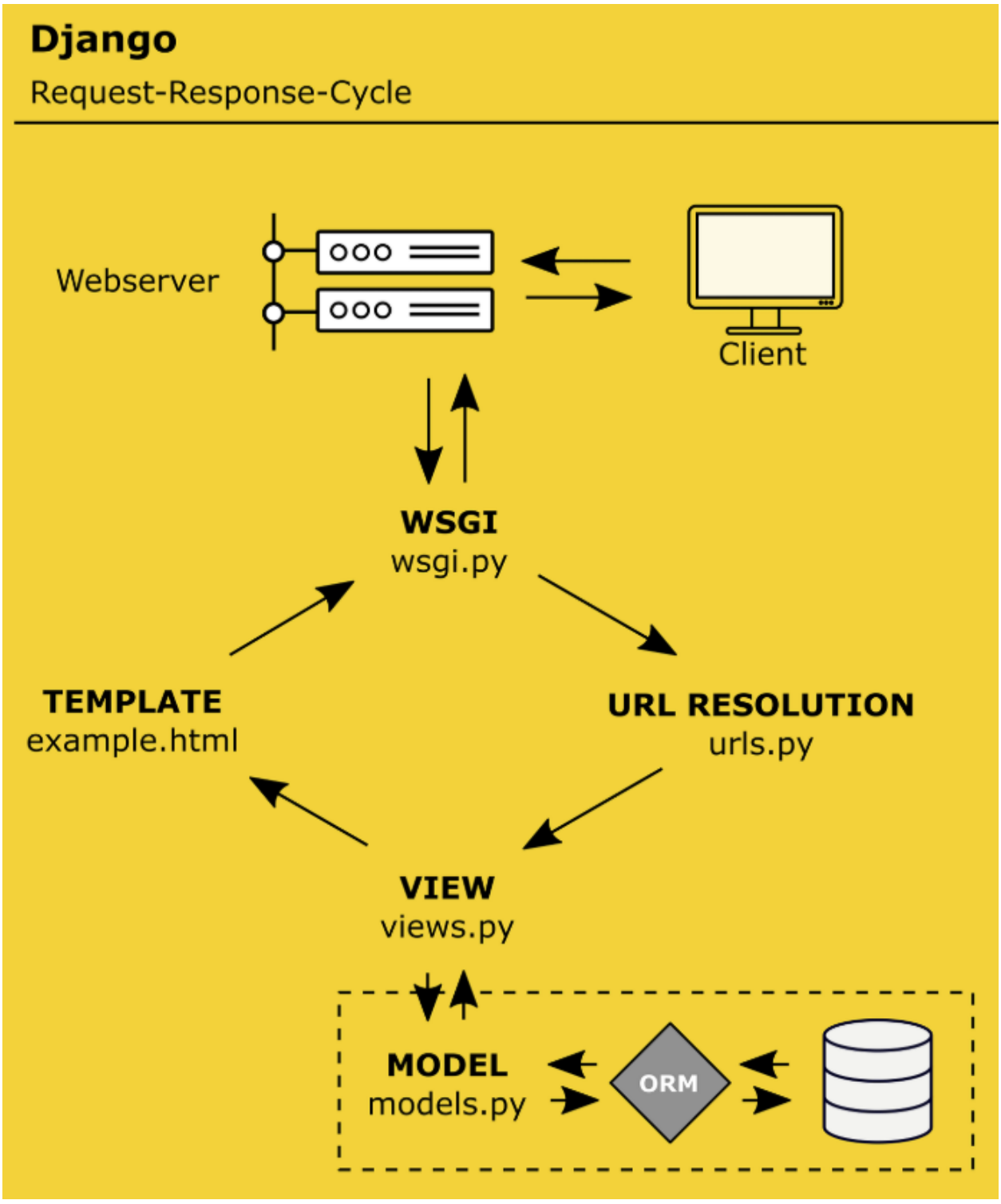
Widgets attribute is python dictionary mapping keys and values.

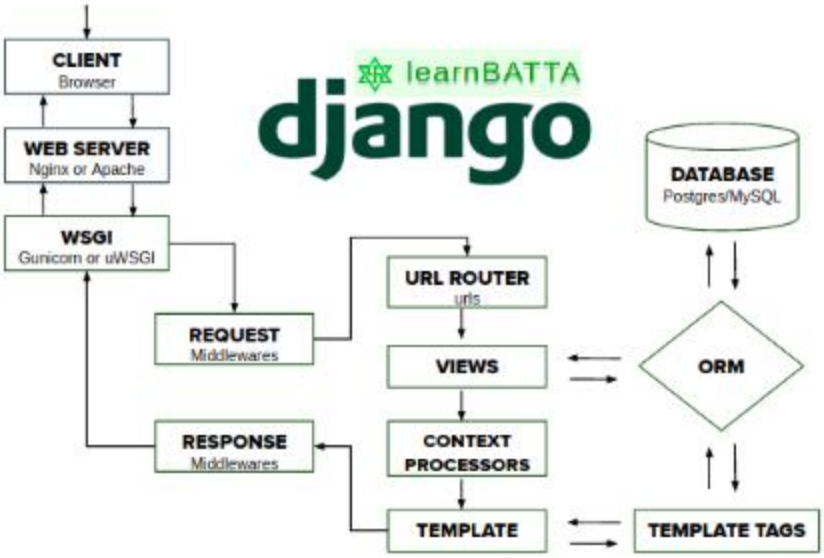
Object instantiation:

In the views file, it creates form objects, which creates an instance of the form class.

```
# This function will show Create Character page when requested
def create_zelda_character(request):  1 usage  Erica Millan
    form = CharacterForm(data=request.POST or None)

    if request.method == 'POST':
        if form.is_valid():
            character = form.save(commit=False)
            character.more_characters_list = False   # <-- Characters created on the site will se
```


webserver recieves http request forrwars to WSGI
Middleware
Internet
User
Browser
HTTP request
TEMPLATE ENGINE
Example.html
URLS.PY
URL routing
FORM.PY
Query & Data
MODELS.PY
Query & Data
ORM
no
yes
needs to acces model or forms?
yes
uses model and form?
no
Model only?
yes
Form only
yes
FORM.PY
Query & Data
VIEWS.PY
Controller


Django
Request-Response-Cycle
Webserver
Client
WSGI
wsgi.py
URL RESOLUTION
urls.py
VIEW
views.py
MODEL
models.py
ORM
TEMPLATE
example.html




request-response lifecycle in Django